

HEART DISEASE PREDICTION – MLOPS PIPELINE

Assignment 1 - MLOps

Abstract

This project develops an end-to-end machine learning workflow for predicting heart disease risk using patient clinical attributes. The solution covers data processing, model development, experiment tracking, automated testing, containerization, Kubernetes deployment, and monitoring.

Git Repo: <https://github.com/alphyelsa/heart-disease-mlops>

Alphy Elsa Sebastian

2024ad05327@wilp.bits-pilani.ac.in

Contents

1. Introduction & System Overview	3
1.1. Objective	3
1.2. Problem Definition	3
1.3. System Architecture Overview	3
1.3.1. Data Acquisition and Processing	3
1.3.2. Feature Engineering and Model Development	3
1.3.3. Experiment Tracking	4
1.3.4. Model Packaging and Deployment	4
1.3.5. Production Deployment and Monitoring	4
1.4. Technologies Used	4
1.5. Architecture Workflow	4
2. Data Acquisition & Exploratory Data Analysis (EDA)	6
2.1. Dataset Acquisition	6
2.2. Data Cleaning and Preprocessing	6
2.3. Exploratory Data Analysis (EDA)	7
2.3.1. Class Distribution Analysis	7
2.3.2. Correlation Analysis	8
2.3.3. Feature Distribution Analysis	9
3. Feature Engineering & Model Development	11
3.1. Feature Engineering and Preprocessing Pipeline	11
3.2. Machine Learning Models	11
3.2.1. Logistic Regression	11
3.2.2. Random Forest Classifier	12
3.3. Model Training Process	12
3.4. Cross Validation and Model Evaluation	13
4. Experiment Tracking with MLflow	14
4.1. MLflow Experiment Tracking Overview	14
4.2. MLflow Tracking Workflow	15
5. Model Packaging & Reproducibility	17
5.1 Model Packaging	17
5.2 Project Structure	17

5.3 Dependency Management.....	18
5.4. Summary	19
6. CI/CD Pipeline & Automated Testing	20
6.1 Continuous Integration and Continuous Deployment (CI/CD)	20
6.2 Automated Testing	21
7. Model Containerization	22
7.1. API Input and Output	22
7.2. Docker Containerization Workflow	22
7.3. Docker Build Verification.....	23
7.4. Running the FastAPI Docker Container	23
7.5. API Testing and Validation.....	23
7.6. Summary	24
8. Production Deployment	25
8.1. Kubernetes Deployment	25
8.2. Kubernetes Deployment Flow.....	25
8.3. Kubernetes Components Used	26
8.4. Deployment Verification	26
9. Monitoring & Logging	27
9.1. Monitoring Overview	27
9.2. Prometheus Monitoring.....	27
9.3. Grafana Dashboard	28
10. Conclusion & Future Improvements	29

1. Introduction & System Overview

1.1. Objective

The objective of this project is to design and implement a complete MLOps pipeline for predicting the presence of heart disease using machine learning techniques. The project focuses on developing a reliable, reproducible, and production-ready machine learning system that covers the complete ML lifecycle, including data processing, model development, experiment tracking, deployment, and monitoring.

The system solves a binary classification problem, where the goal is to predict whether a patient has a risk of heart disease or not. The target variable represents the presence of heart disease, with the model output indicating the predicted class along with confidence information.

1.2. Problem Definition

Heart disease prediction is formulated as a supervised binary classification task. Given patient health-related features such as age, blood pressure, cholesterol level, and other clinical attributes, the machine learning model learns patterns from historical data and predicts the likelihood of heart disease.

The dataset used in this project is the UCI Heart Disease Dataset, which contains medical attributes and corresponding target labels used for training and evaluating classification models.

1.3. System Architecture Overview

The developed architecture follows a complete machine learning lifecycle:

1.3.1. Data Acquisition and Processing

The heart disease dataset is collected and prepared for analysis. Data cleaning is performed to handle missing values, remove inconsistencies, and prepare the dataset for modeling. Exploratory Data Analysis (EDA) is performed using statistical analysis and visualizations to understand feature distributions, correlations, and class balance.

1.3.2. Feature Engineering and Model Development

A feature processing pipeline is created using preprocessing techniques such as scaling and encoding. Two classification models are developed and evaluated: Logistic Regression, Random Forest Classifier. Model performance is compared using evaluation metrics such as accuracy, precision, recall, and ROC-AUC.

1.3.3. Experiment Tracking

MLflow is integrated to track machine learning experiments. Parameters, evaluation metrics, model artifacts, and visualization outputs are logged for reproducibility and comparison.

1.3.4. Model Packaging and Deployment

The best-performing model is saved as a reusable model artifact. The model is integrated with a FastAPI application to provide real-time prediction capabilities. Docker is used to package the API, model, and dependencies into a portable container.

1.3.5. Production Deployment and Monitoring

The Dockerized FastAPI application is deployed using Kubernetes/Minikube. Kubernetes manages application availability through deployments and services. Prometheus and Grafana are used for monitoring API performance, metrics collection, and visualization.

1.4. Technologies Used

Component	Technology
Data Processing	Pandas, Scikit-learn
Machine Learning Models	Logistic Regression, Random Forest
Experiment Tracking	MLflow
Testing	Pytest
CI/CD Pipeline	GitHub Actions
API Development	FastAPI
Containerization	Docker
Deployment	Kubernetes / Minikube
Monitoring	Prometheus + Grafana

1.5. Architecture Workflow

The overall workflow of the system is:

Dataset → Data Cleaning & EDA → Feature Pipeline → Model Training → MLflow Tracking → Best Model Artifact → FastAPI Docker Container → Kubernetes Deployment → Monitoring

The architecture enables a complete automated ML workflow where data preparation, model training, testing, deployment, and monitoring are connected together to support reliable machine learning operations.

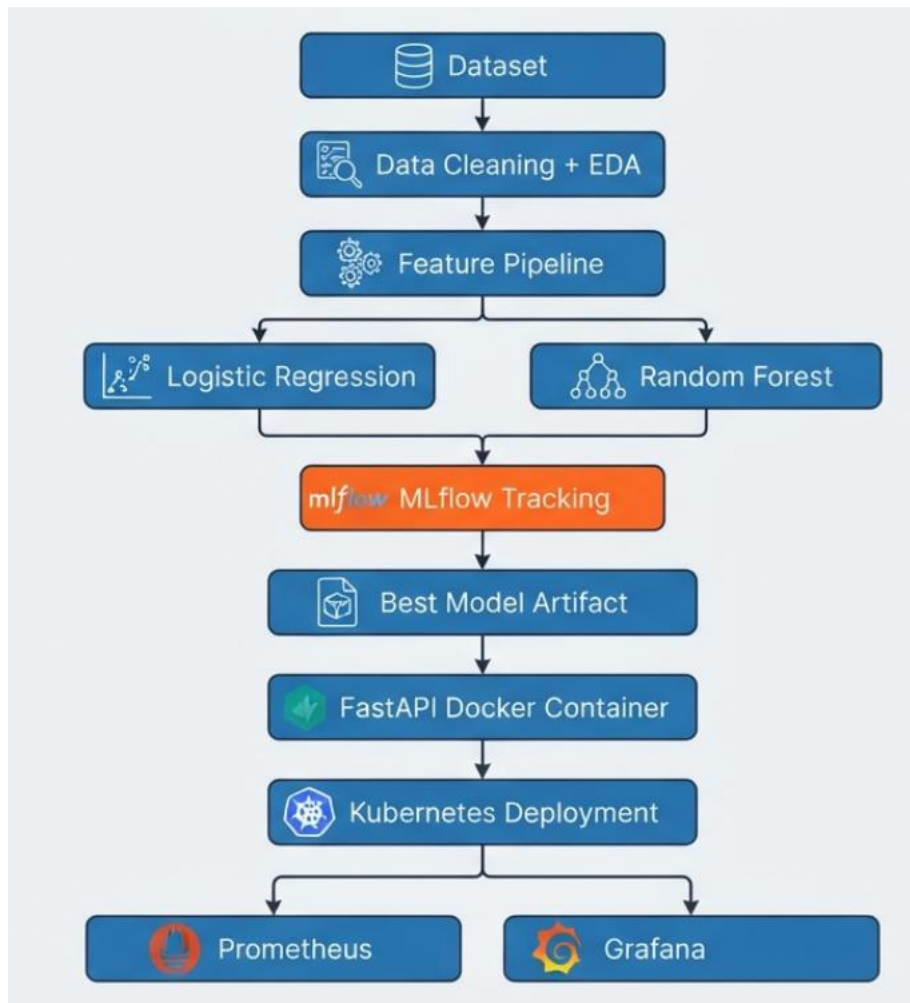


Figure 1: Overall MLOps architecture for the heart disease prediction system.

2. Data Acquisition & Exploratory Data Analysis (EDA)

2.1. Dataset Acquisition

The heart disease prediction system uses the UCI Heart Disease Dataset, which contains clinical patient information used to train and evaluate machine learning classification models. The objective of using this dataset is to predict the presence of heart disease based on patient medical attributes.

The dataset contains multiple clinical features, including:

- **Age** – Patient age
- **Sex** – Gender information
- **Chest Pain Type (cp)** – Type of chest pain experienced
- **Blood Pressure (trestbps)** – Resting blood pressure
- **Cholesterol (chol)** – Serum cholesterol level
- **Maximum Heart Rate (thalach)** – Maximum achieved heart rate
- **Exercise-related and clinical measurements**
- **Target** – Heart disease presence label

The dataset acquisition process is automated using the `download_data.py` script. The script downloads the dataset and stores it in the project directory under: `data/raw`. This ensures that the data collection process is reproducible and can be executed automatically during pipeline execution. The dataset can be downloaded using the following command:

```
python src/download_data.py
```

```
(.venv) PS D:\Github\heart-disease-mlops> python .\src\download_data.py
Downloading dataset from https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data...
  age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  target
0  63.0  1.0  1.0    145.0  233.0  1.0     2.0    150.0    0.0     2.3    3.0  0.0    6.0     0
1  67.0  1.0  4.0    160.0  286.0  0.0     2.0    108.0    1.0     1.5    2.0  3.0    3.0     2
2  67.0  1.0  4.0    120.0  229.0  0.0     2.0    129.0    1.0     2.6    2.0  2.0    7.0     1
3  37.0  1.0  3.0    130.0  250.0  0.0     0.0    187.0    0.0     3.5    3.0  0.0    3.0     0
4  41.0  0.0  2.0    130.0  204.0  0.0     2.0    172.0    0.0     1.4    1.0  0.0    3.0     0
Dataset securely saved to data/raw/heart.csv
```

Figure 2: Preview of the heart disease dataset showing patient attributes and target labels.

2.2. Data Cleaning and Preprocessing

Before model development, the raw dataset was cleaned and prepared to ensure data quality and consistency. The preprocessing stage includes handling missing values, correcting data types, checking duplicate records, and separating input features from the prediction target.

The following cleaning steps were performed:

Missing Value Handling: The dataset was checked for missing or incomplete records. Appropriate handling techniques were applied to ensure that the machine learning models receive complete input data.

Data Type Conversion: All features were reviewed and converted into suitable numerical formats required by machine learning algorithms.

Duplicate Record Checking: Duplicate entries were identified and removed to prevent biased model training caused by repeated samples.

Feature and Target Separation: The cleaned dataset was divided into input features and target labels.

```
Checking null values...
age          0
sex          0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
target      0
dtype: int64
-----
```

Figure 2: Missing value analysis confirming data completeness after preprocessing.

2.3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the characteristics of the dataset, identify important relationships between variables, and analyse the distribution of the target classes.

EDA helps in understanding patient characteristics and selecting appropriate preprocessing and modeling strategies.

2.3.1. Class Distribution Analysis

The distribution of heart disease and non-heart disease cases was analysed to determine whether the dataset contains balanced classes. The class distribution analysis shows the proportion of patients with and without heart disease. A balanced distribution helps prevent the model from becoming biased toward one class and improves classification reliability.

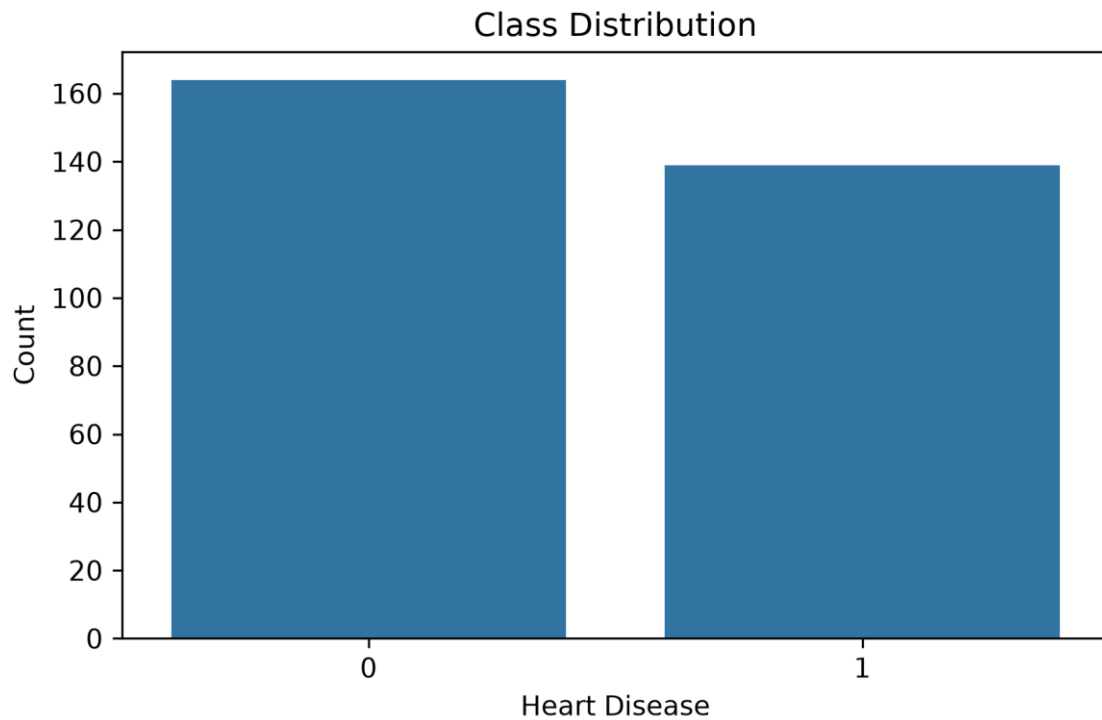


Figure 4: Distribution of heart disease and non-heart disease classes.

2.3.2. Correlation Analysis

A correlation heatmap was generated to identify relationships between clinical features and understand which variables have stronger associations with the target variable.

The heatmap helps identify:

- Features with strong positive or negative relationships
- Potentially important predictors
- Relationships between clinical measurements

Important variables such as cholesterol level, maximum heart rate, age, and chest pain type were analyzed to understand their contribution to heart disease prediction.

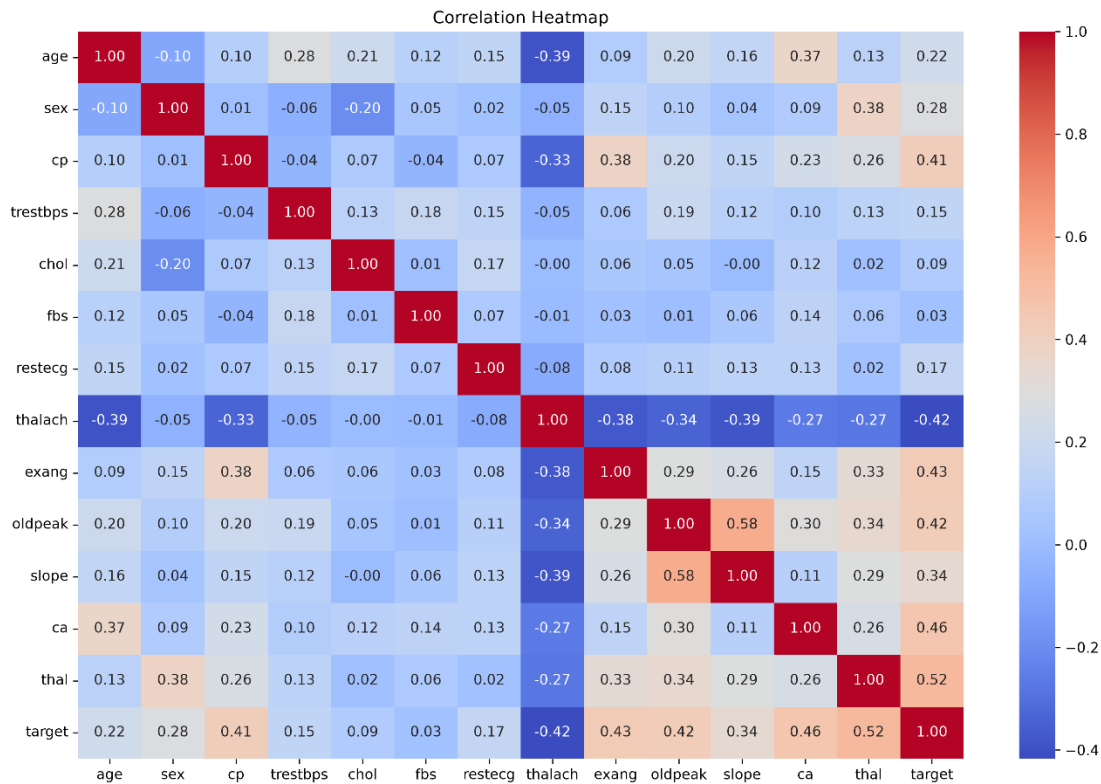


Figure 5: Correlation heatmap showing relationships between clinical features and heart disease target.

2.3.3. Feature Distribution Analysis

Feature distributions were analyzed using visualizations such as histograms to understand the range, spread, and patterns of important patient attributes.

The following distributions were examined:

- Age distribution
- Cholesterol distribution
- Maximum heart rate distribution

These visualizations help identify unusual patterns, outliers, and the overall characteristics of patient data.

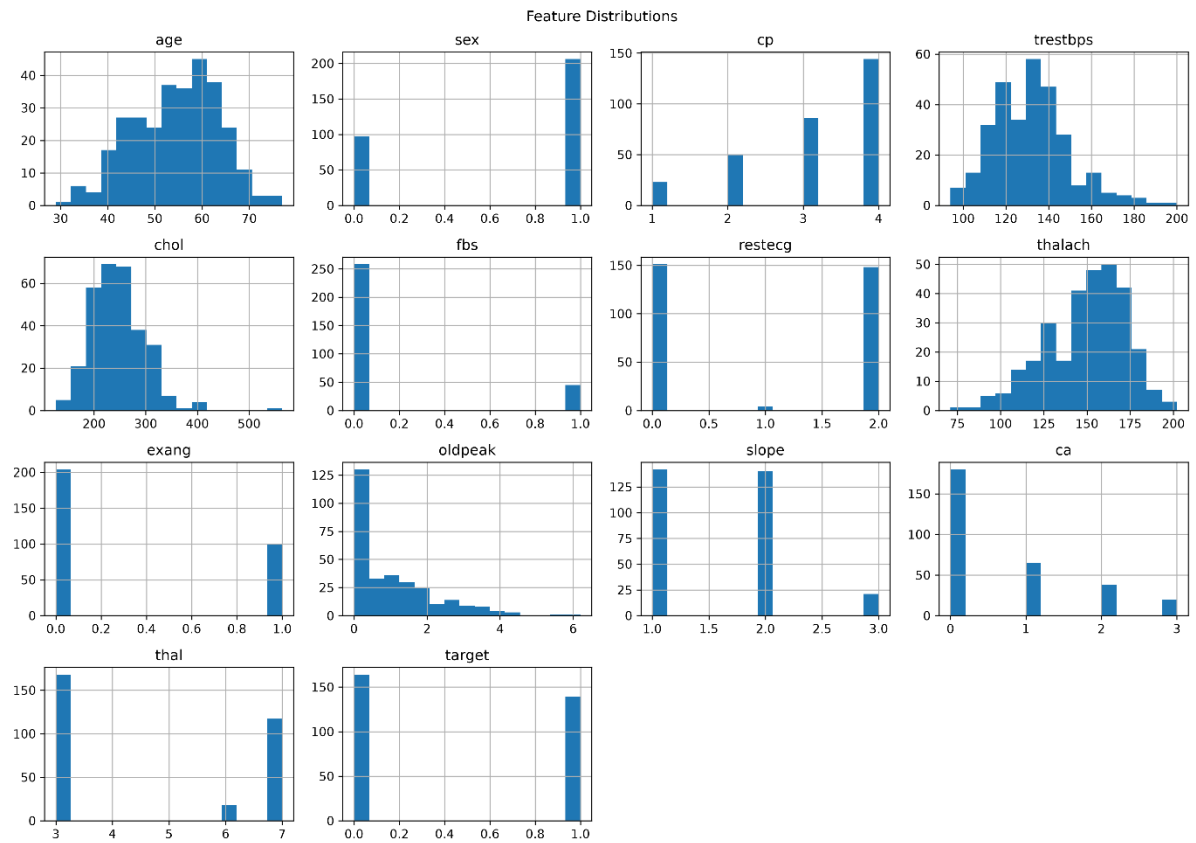
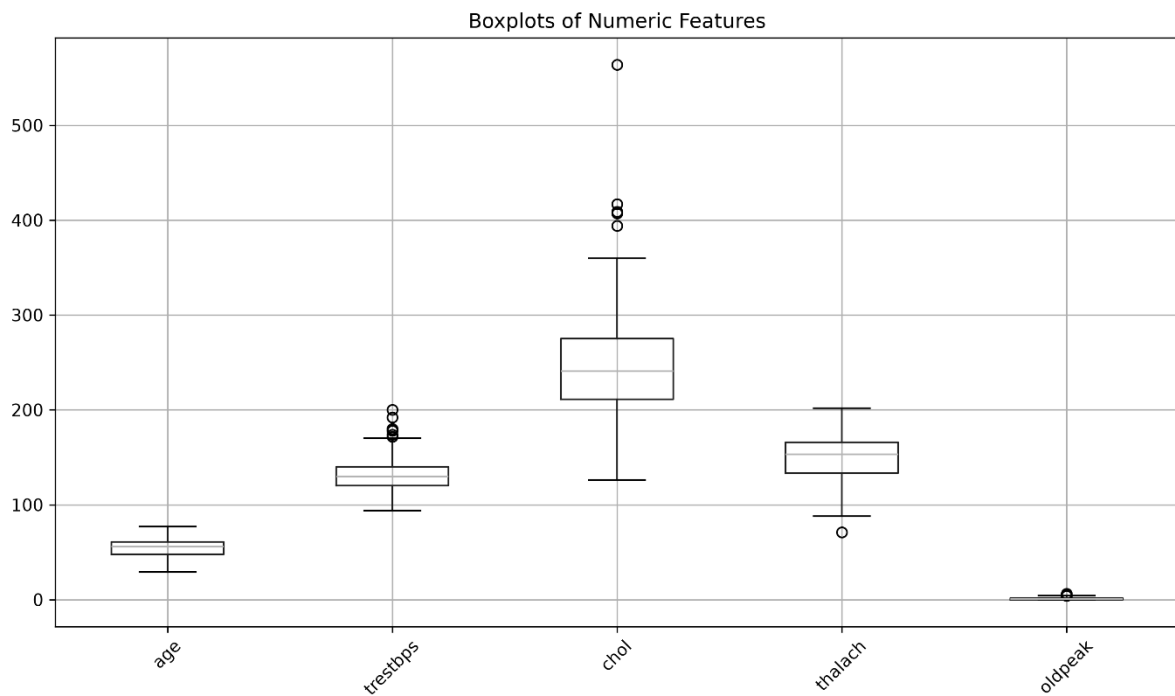


Figure 6: Distribution of important clinical features used for heart disease prediction.



3. Feature Engineering & Model Development

3.1. Feature Engineering and Preprocessing Pipeline

Feature engineering is an important step in preparing the heart disease dataset for machine learning model training. The purpose of preprocessing is to transform raw patient information into a suitable format that can be effectively used by classification algorithms.

A complete preprocessing pipeline was developed using Scikit-learn Pipeline to ensure that all data transformation steps are applied consistently during training and prediction.

The preprocessing workflow is: **Raw Features → Missing Value Handling → Feature Scaling → Machine Learning Model**

The preprocessing pipeline performs the following operations:

1. **Missing Value Handling:** Identifies and processes incomplete records and ensures that the model receives complete and valid input data.
2. **Feature Scaling:** Numerical features are scaled to bring different variables into a similar range. Scaling improves model performance, especially for algorithms sensitive to feature magnitude such as Logistic Regression.
3. **Pipeline Integration:** The preprocessing steps and model training process are combined into a single pipeline. This improves reproducibility by ensuring that the same transformations are applied during both training and future predictions.

3.2. Machine Learning Models

Two classification models were developed and evaluated for predicting heart disease risk.

3.2.1. Logistic Regression

Logistic Regression was selected as a baseline classification model because it provides a simple and interpretable approach for binary classification problems.

Advantages include

- Provides a strong baseline for comparison.
- Produces interpretable coefficients that show the relationship between input features and prediction outcomes.
- Performs well when relationships between variables are approximately linear.

The model predicts the probability of heart disease presence based on the input clinical features.

3.2.2. Random Forest Classifier

Random Forest was selected as a second model because it can capture complex relationships within the dataset.

Advantages:

- Handles nonlinear relationships between features.
- Robust against feature interactions.
- Reduces overfitting by combining multiple decision trees.
- Performs well on structured medical datasets.

The Random Forest model learns patterns from multiple decision trees and combines their predictions to improve classification performance.

3.3. Model Training Process

The dataset was divided into training and testing sets to evaluate model performance on unseen patient data. The data split was performed as follows: Training Data: 80%, Testing Data: 20%

A fixed random seed was used to ensure that experiments are reproducible and produce consistent results. The training dataset was used to build the models, while the testing dataset was reserved for final performance evaluation.

```
D:\Github\heart-disease-mlops\.venv\Scripts\python.exe D:\Github\heart-disease-mlops\src\train.py

=====
MODEL TRAINING STARTED
=====
Training time: 2026-07-11 21:46:08.064338
Training samples: 242
Testing samples: 61

-----
Training model: LogisticRegression
-----
LogisticRegression completed successfully
ROC-AUC: 0.9159

-----
Training model: RandomForest
-----
RandomForest completed successfully
ROC-AUC: 0.9472
```

Figure 7: Successful training and experiment logging of Logistic Regression and Random Forest models.

3.4. Cross Validation and Model Evaluation

To obtain reliable performance estimates, 5-fold cross validation was applied during model evaluation.

In 5-fold cross validation:

1. The dataset is divided into five equal sections.
2. The model is trained on four sections and validated on the remaining section.
3. This process is repeated five times.
4. The final performance score is calculated using the average results.

The following evaluation metrics were used:

Accuracy: Measures the overall percentage of correct predictions.

Precision: Measures how many predicted heart disease cases were actually positive.

Recall: Measures the ability of the model to correctly identify patients with heart disease.

ROC-AUC: Measures the model's ability to distinguish between positive and negative classes across different classification thresholds.

These metrics provide a comprehensive evaluation of model performance, especially important for medical prediction problems.

```
=====
MODEL PERFORMANCE COMPARISON
=====
      Model  Accuracy  Precision  Recall  ROC-AUC
LogisticRegression    0.8361    0.8438    0.8438    0.9159
      RandomForest    0.8689    0.9000    0.8438    0.9472

Best model selected:
RandomForest
```

Figure 8: Performance comparison between Logistic Regression and Random Forest models.

4. Experiment Tracking with MLflow

4.1. MLflow Experiment Tracking Overview

To ensure reproducibility and effective management of machine learning experiments, MLflow was integrated into the heart disease classification pipeline. MLflow provides a centralized platform for tracking model development activities, comparing different experiments, and managing trained model artifacts.

The experiment tracking system records important information from every model training run, including:

Parameters: Model configuration values used during training.

Examples: Random Forest number of estimators, Maximum tree depth, Learning settings

Metrics: Performance measurements used to evaluate models.

Examples: Test accuracy, Test precision, Test recall, ROC-AUC score

Models: Stores the trained machine learning models in a reusable format and enables model loading and deployment without retraining.

Visual Artifacts: Stores generated evaluation plots and analysis files.

Examples: Confusion matrix, ROC curve, Precision-Recall curve

The MLflow experiment created for this project is: `Heart_Disease_Classification`. The experiment contains multiple training runs: `LogisticRegression` and `RandomForest`. Each run contains the model configuration, evaluation metrics, and generated artifacts, allowing direct comparison between different approaches.

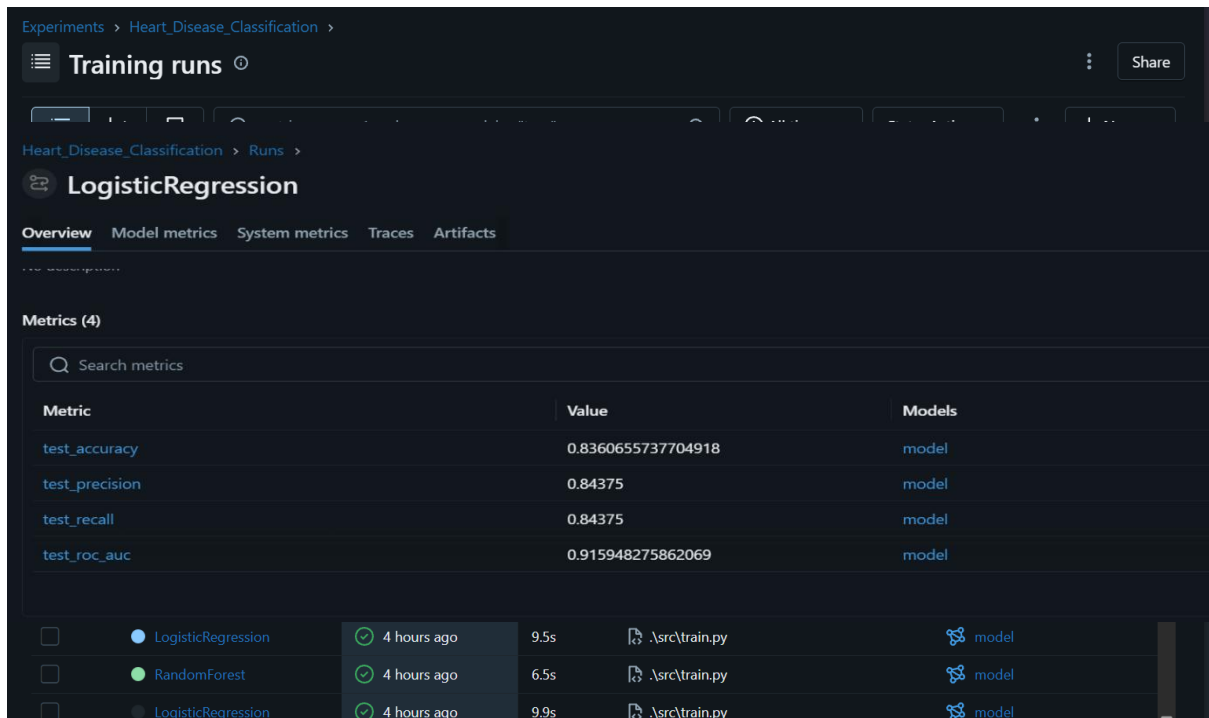
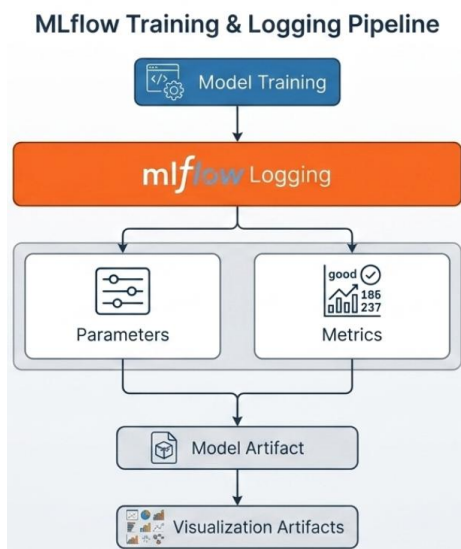


Figure 8: MLflow experiment dashboard showing tracked Logistic Regression and Random Forest runs.

4.2. MLflow Tracking Workflow

The experiment tracking workflow follows these steps:



During model training, MLflow automatically records each experiment run. This makes the training process reproducible and helps identify the best-performing model for deployment.

Figure 10: MLflow run details showing model parameters and evaluation metrics.

Heart_Disease_Classification > Runs >

LogisticRegression

Overview Model metrics System metrics Traces **Artifacts**

- MLmodel
- conda.yaml
- model.skops
- python_env.yaml
- requirements.txt

MLmodel 769B

Path: file:///D:/Github/heart-disease-mlops/src/mlruns/704742787323719905/e5705eae0d9548fea9e118093e1d3052/...

artifact_path: file:///D:/Github/heart-disease-mlops/src/mlruns/704742787323719905/models/m-801437314

flavors:

```
python_function:
  env:
    conda: conda.yaml
    virtualenv: python_env.yaml
  loader_module: mlflow.sklearn
  model_path: model.skops
  predict_fn: predict
  python_version: 3.12.10
sklearn:
  code: null
  pickled_model: model.skops
  serialization_format: skops
  sklearn_version: 1.9.0
  skops_trusted_types:
    - numpy.dtype
mlflow_version: 3.14.0
model_id: m-8014373145cf40c485cb210f33215cef
model_size_bytes: 199858
```

Figure 11: MLflow artifacts containing model files and other related files.

5. Model Packaging & Reproducibility

5.1 Model Packaging

After evaluating the performance of the classification models, the best-performing model was selected and exported as a reusable model artifact. Model packaging ensures that the trained model can be easily loaded for prediction without requiring retraining, making deployment and future reuse more efficient.

The final model is stored using the MLflow model format (or another serialized format such as model.pkl, depending on the implementation). The packaged model includes not only the trained classifier but also the associated metadata required for deployment and reproducibility. (Refer *Figure 11: MLflow artifacts containing model files and other related files*)

The packaged model contains:

- **Preprocessing pipeline** – Stores the preprocessing steps (such as scaling and feature transformations) applied during training, ensuring that new input data is processed consistently before prediction.
- **Trained classifier** – Contains the final machine learning model selected based on evaluation metrics.
- **Environment information** – Includes dependency and environment details required to reproduce the model in different systems.

Packaging the complete pipeline guarantees that the same preprocessing and prediction workflow is applied during both development and deployment, improving consistency and reproducibility.

5.2 Project Structure

The project follows a modular directory structure to separate source code, trained models, tests, configuration files, and deployment resources. This organization improves maintainability, readability, and collaboration.

A typical project structure is shown below:

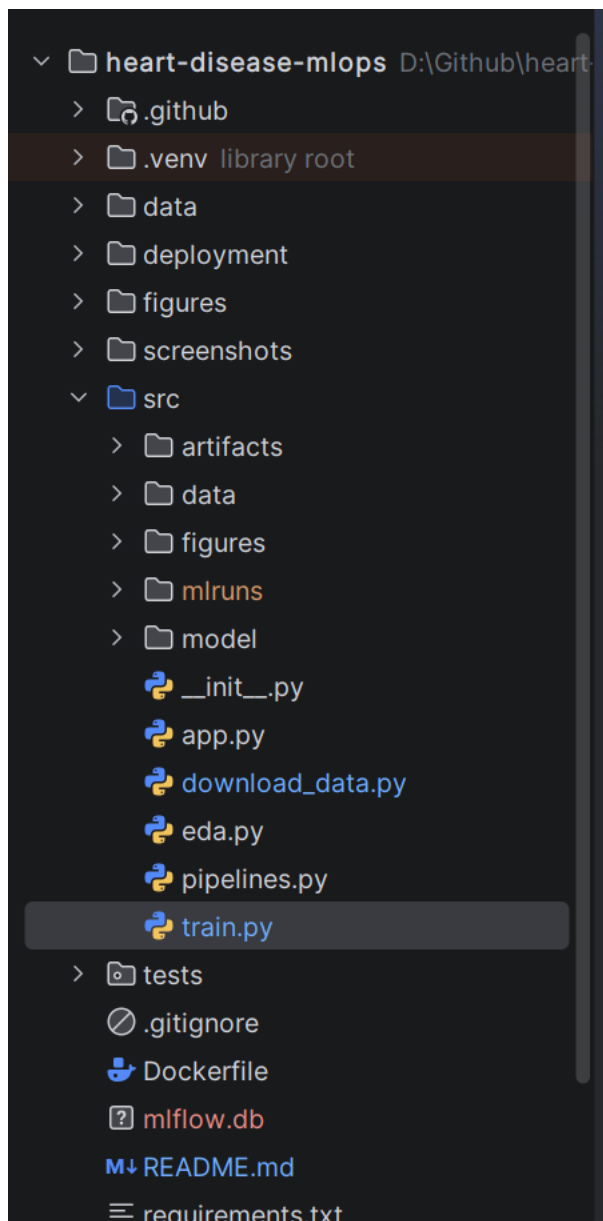
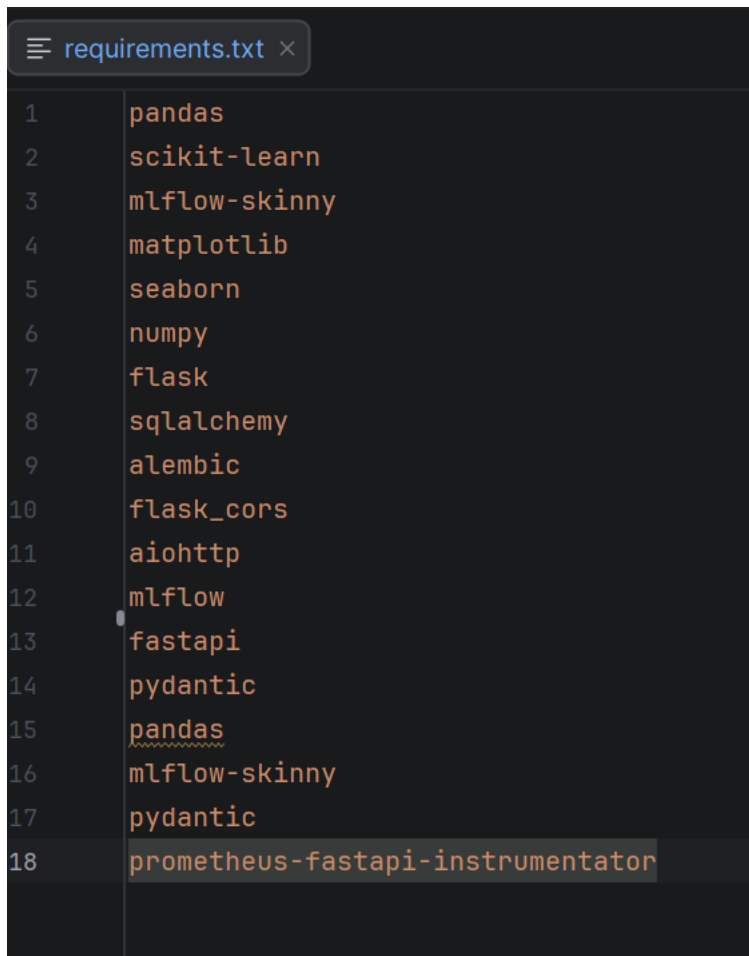


Figure 12: Project directory structure of the heart disease MLOps pipeline.

5.3 Dependency Management

To ensure reproducibility across different environments, all required Python libraries and package versions are listed in the requirements.txt file. This file enables consistent installation of dependencies during development, testing, and deployment.

A screenshot of a code editor window with a dark theme. The title bar at the top shows a hamburger menu icon, the filename 'requirements.txt', and a close button. The editor contains 18 lines of text, each a package name. Lines 15 and 16 have red squiggly lines under the package names 'pandas' and 'mlflow-skinny' respectively. Line 18 is highlighted with a light blue background. The packages listed are: pandas, scikit-learn, mlflow-skinny, matplotlib, seaborn, numpy, flask, sqlalchemy, alembic, flask_cors, aiohttp, mlflow, fastapi, pydantic, pandas, mlflow-skinny, pydantic, and prometheus-fastapi-instrumentator.

```
1 pandas
2 scikit-learn
3 mlflow-skinny
4 matplotlib
5 seaborn
6 numpy
7 flask
8 sqlalchemy
9 alembic
10 flask_cors
11 aiohttp
12 mlflow
13 fastapi
14 pydantic
15 pandas
16 mlflow-skinny
17 pydantic
18 prometheus-fastapi-instrumentator
```

Figure 13: Project dependencies specified in the requirements.txt file.

Developers can recreate the project environment using:

```
pip install -r requirements.txt
```

Maintaining a complete dependency list helps prevent compatibility issues and ensures that the packaged model behaves consistently across different machines.

5.4. Summary

The final machine learning model was packaged as a reusable artifact together with its preprocessing pipeline and environment information, enabling consistent predictions and straightforward deployment. A well-organized project structure and a comprehensive requirements.txt file further improve reproducibility by ensuring that the project can be recreated and executed reliably across different development and production environments.

6. CI/CD Pipeline & Automated Testing

6.1 Continuous Integration and Continuous Deployment (CI/CD)

To automate software quality assurance and machine learning workflows, a Continuous Integration and Continuous Deployment (CI/CD) pipeline was implemented using GitHub Actions. The pipeline automatically executes whenever code is pushed to the project repository, ensuring that every change is validated before deployment.

The automated workflow performs several stages, including dependency installation, code quality checks, unit testing, model training, and artifact generation. This automation reduces manual effort, improves reproducibility, and helps identify issues early during development.

The overall CI/CD workflow is illustrated below: **Push Code → Install Dependencies → Linting → Unit Tests → Model Training → Upload Artifacts**

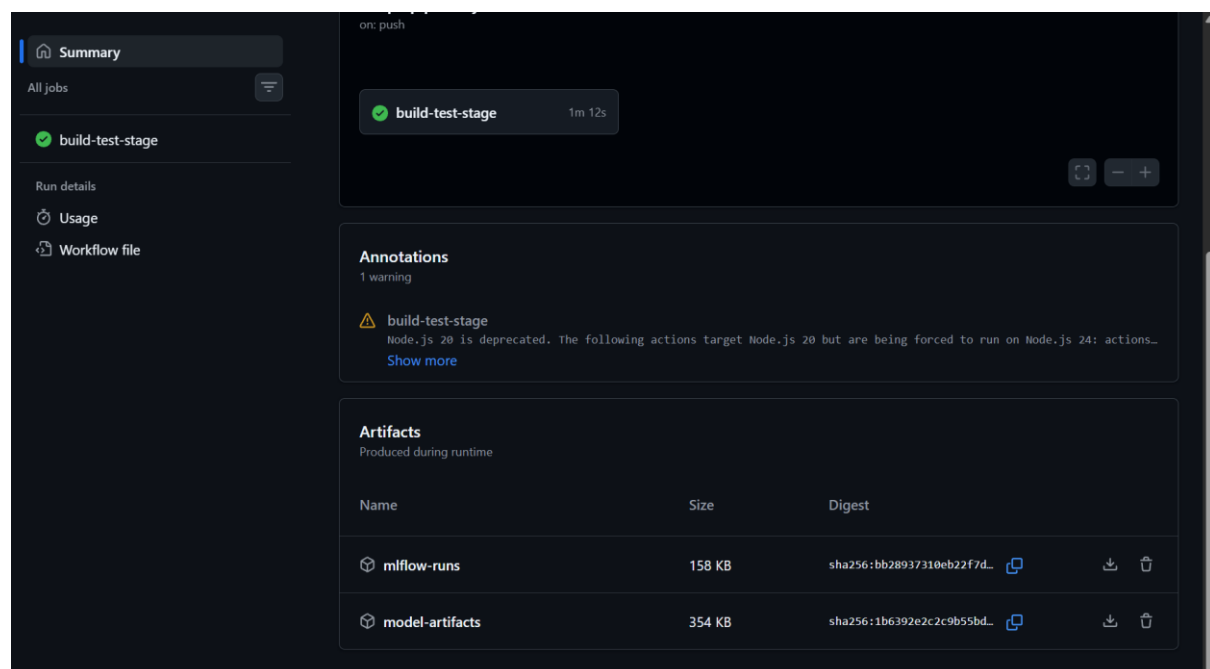


Figure 13: Successful GitHub Actions workflow showing dependency installation, linting, testing, model training, and artifact upload.

The pipeline executes the following stages:

1. Install Dependencies

- Required Python libraries are installed using the project's dependency file.
- This ensures a consistent execution environment for every workflow run.

2. Linting

- Source code is checked for syntax errors and coding style issues.
- Maintaining code quality improves readability and reduces potential bugs.

3. Unit Testing

- Automated tests verify that critical components of the machine learning pipeline function correctly before model training begins.

4. Model Training

- Once all tests pass, the machine learning models are trained automatically.
- This guarantees that only validated code is used to generate new model artifacts.

5. Artifact Upload

- The trained model and generated experiment logs are stored as workflow artifacts.
- These artifacts can be downloaded later for deployment or further analysis.

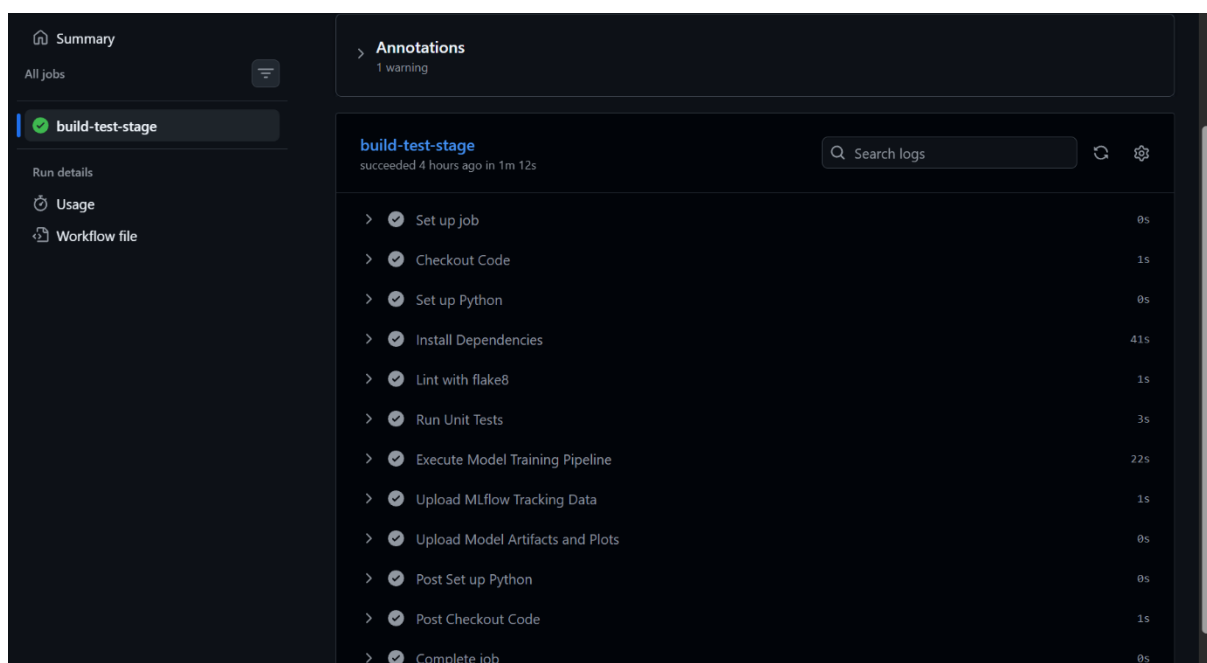


Figure 14: GitHub Actions artifacts generated during the CI/CD workflow, including the trained model and MLflow logs.

6.2 Automated Testing

Automated unit tests were developed to validate the correctness of the preprocessing pipeline and machine learning components. These tests ensure that future code modifications do not introduce unexpected errors.

By running these tests automatically during every GitHub Actions workflow, the project maintains consistent data quality and reliable model behavior.

7. Model Containerization

The trained heart disease prediction model was deployed as a REST API using FastAPI to enable real-time predictions. FastAPI provides a lightweight and efficient interface for serving machine learning models by exposing prediction endpoints that can receive patient information and return prediction results.

The API provides a POST /predict endpoint that accepts patient health features in JSON format. The input data is processed using the same preprocessing pipeline used during model training, and the trained model generates a prediction along with a confidence score.

7.1. API Input and Output

The prediction endpoint accepts JSON formatted patient features as input. After processing the input through the ML pipeline, the API returns the predicted heart disease risk and model confidence value.

Example response:

```
{  
  "heart_disease_risk": 1,  
  "confidence": 0.87  
}
```

Where:

- **heart_disease_risk = 1** indicates a higher risk of heart disease.
- **confidence = 0.87** represents the model confidence score for the prediction.

7.2. Docker Containerization Workflow

To ensure portability and reproducibility, the FastAPI application and trained model were packaged into a Docker container. Docker provides a consistent runtime environment by including the application code, dependencies, preprocessing pipeline, and model artifact.

The containerization workflow follows the process:

Model Artifact → Docker Image → FastAPI Container

1. The trained machine learning model is saved as a reusable model artifact.
2. The model artifact, FastAPI application, and required dependencies are packaged into a Docker image.
3. The Docker image is executed as a container, exposing the prediction API through port 8000.

4. The running container provides access to the /predict endpoint for real-time predictions.

```
(.venv) PS D:\Github\heart-disease-mlops> docker build -t heart-disease-api .
[*] Building 1.9s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 682B
=> [internal] load metadata for docker.io/library/python:3.10-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 149.12kB
=> [builder 1/5] FROM docker.io/library/python:3.10-slim@sha256:e5300dc020a26a34a19337a57602955a2510e22abeb176edd6de6cd2cc927dd4
=> => resolve docker.io/library/python:3.10-slim@sha256:e5300dc020a26a34a19337a57602955a2510e22abeb176edd6de6cd2cc927dd4
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] RUN apt-get update && apt-get install -y --no-install-recommends build-essential && rm -rf /var/lib/apt/lists/*
=> CACHED [builder 4/5] COPY requirements.txt .
=> CACHED [builder 5/5] RUN pip install --user --no-cache-dir -r requirements.txt
=> CACHED [runner 3/6] COPY --from=builder /root/.local /root/.local
=> CACHED [runner 4/6] COPY ./src ./src
=> CACHED [runner 5/6] COPY ./src/mlruns ./mlruns
=> CACHED [runner 6/6] COPY ./model ./model
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:7be5537157d9affc1ef76157947464350699392cc5868ed83039708890da89b
=> => exporting config sha256:7e3ae542195680413dccc47b08b6c33df14d9a2c78f59766e9dd6661f8fe4025
=> => exporting attestation manifest sha256:473e4d45f6b30dd03ed51f27c9d06907d5c667c48b6a1d28b6d06b8af68f245a
=> => exporting manifest list sha256:3f01a72aeea04007baa8eb74224964391a62eefbf95794571ae3084b58052226
=> => naming to docker.io/library/heart-disease-api:latest
=> => unpacking to docker.io/library/heart-disease-api:latest
```

Figure 15: Successful Docker image build for the heart disease prediction API.

7.3. Docker Build Verification

The Docker image was successfully created using the project Dockerfile. The successful build confirms that all required dependencies, application files, and model artifacts were correctly packaged.

7.4. Running the FastAPI Docker Container

The Docker container was started locally to verify that the API service runs correctly in a containerized environment. The container maps port 8000 from the host machine to port 8000 inside the container, allowing access to the FastAPI service.

```
(.venv) PS D:\Github\heart-disease-mlops> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
914842d35369   heart-disease-api                  "uvicorn src.app:app..." 23 seconds ago Up 23 seconds 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
5f7353516cd2   gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 6 hours ago    Up 6 hours    127.0.0.1:61522->22/tcp, 127.0.0.1:61519->32443/tcp, 127.0.0.1:61520->5000/tcp, 127.0.0.1:61521->8443/tcp, 127.0.0.1:61518->2376/tcp, 127.0.0.1:61520->5000/tcp, 127.0.0.1:61521->8443/tcp, 127.0.0.1:61519->32443/tcp, 127.0.0.1:61518->2376/tcp
```

Figure 16: Running FastAPI Docker container with port 8000 exposed.

7.5. API Testing and Validation

The deployed FastAPI service was tested to verify that the prediction endpoint works correctly. FastAPI automatically generates interactive API documentation using Swagger UI, which allows users to submit test inputs and view API responses.

The Swagger interface was accessed through:

<http://localhost:8000/docs>

A sample patient JSON input was submitted to the /predict endpoint, and the API successfully returned the heart disease prediction and confidence score.

Alternative testing can also be performed using a curl request:

```
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{"age":55,"cholesterol":240,"blood_pressure":140}'
```

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "age": 35,
    "sex": 1,
    "cp": 2,
    "trestbps": 124,
    "chol": 100,
    "fbs": 1,
    "restecg": 1,
    "thalach": 150,
    "exang": 1,
    "oldpeak": 2.5,
    "slope": 1,
    "ca": 2,
    "thal": 3
  }'
```

Request URL	
http://localhost:8000/predict	
Server response	
Code	Details
200	Response body <pre>{ "heart_disease_risk": 0, "confidence": 0.5073010606627153 }</pre>

Figure 17: Successful heart disease prediction through FastAPI Swagger documentation.

7.6. Summary

The FastAPI-based deployment successfully converted the trained heart disease prediction model into a production-ready REST service. Docker containerization ensured that the model, API code, and dependencies were packaged into a consistent and reproducible environment. The successful Docker build, running container verification, and API testing demonstrate that the machine learning model can provide real-time predictions through a scalable API interface.

8. Production Deployment

8.1. Kubernetes Deployment

The heart disease prediction API was deployed using Kubernetes to provide a scalable and reliable production environment. Kubernetes manages the deployment, availability, and networking of the containerized machine learning API. The Docker image containing the trained model and API service was deployed as a Kubernetes Deployment, which ensures that the required number of application instances (replicas) are running continuously.

The deployment was configured with the application name **heart-disease-api** and **two replicas** to improve availability and handle multiple prediction requests. Kubernetes automatically monitors the pods and restarts them if any failure occurs, ensuring the API remains accessible.

```
(.venv) PS D:\Github\heart-disease-mlops> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	2 (48m ago)	3h39m
heart-disease-api-8448bcbb99-cpnr9	1/1	Running	1 (48m ago)	85m
heart-disease-api-8448bcbb99-zksqj	1/1	Running	1 (48m ago)	85m
monitoring-grafana-797f596d9f-lqjrv	3/3	Running	3 (48m ago)	3h40m
monitoring-kube-prometheus-operator-7cc667bdcf-zn6ww	1/1	Running	1 (48m ago)	3h40m
monitoring-kube-state-metrics-6fcd74746d-5v6cw	1/1	Running	1 (48m ago)	3h40m
monitoring-prometheus-node-exporter-4zcdp	1/1	Running	1 (48m ago)	3h40m
prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	2 (48m ago)	3h39m

Figure 18: Kubernetes pods running successfully for the heart disease API deployment.

A Kubernetes Service of type **LoadBalancer** was created to expose the API externally. The service maps external HTTP traffic on **port 80** to the API container, allowing users or external applications to send prediction requests through the generated Kubernetes URL.

```
(.venv) PS D:\Github\heart-disease-mlops> kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP,9094/TCP,9094/UDP	3h41m
heart-disease-service	LoadBalancer	10.107.32.229	<pending>	80:30566/TCP	4h1m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4h20m
monitoring-grafana	ClusterIP	10.107.134.55	<none>	80/TCP	3h42m
monitoring-kube-prometheus-alertmanager	ClusterIP	10.110.31.94	<none>	9093/TCP,8080/TCP	3h42m
monitoring-kube-prometheus-operator	ClusterIP	10.96.197.129	<none>	443/TCP	3h42m
monitoring-kube-prometheus-prometheus	ClusterIP	10.99.157.228	<none>	9090/TCP,8080/TCP	3h42m
monitoring-kube-state-metrics	ClusterIP	10.101.113.169	<none>	8080/TCP	3h42m
monitoring-prometheus-node-exporter	ClusterIP	10.96.238.109	<none>	9100/TCP	3h42m
prometheus-operated	ClusterIP	None	<none>	9090/TCP	3h41m

Figure 19: Kubernetes LoadBalancer service exposing the heart disease API.

8.2. Kubernetes Deployment Flow

The deployment process follows these steps:

Docker Image → Kubernetes Deployment → Service Exposure

1. The trained ML model and API application are packaged into a Docker image.
2. The Docker image is deployed to Kubernetes using a Deployment configuration.
3. Kubernetes creates and manages the required pods running the API.

4. A LoadBalancer Service exposes the API endpoint for external access.
5. Prediction requests can be sent through the exposed URL, and the API returns the prediction result with confidence score.

8.3. Kubernetes Components Used

1. Deployment: heart-disease-api

The Kubernetes Deployment manages the lifecycle of the heart disease API application. It defines the Docker image, number of replicas, and container configuration. Two replicas were configured to ensure high availability and fault tolerance.

2. Service: heart-disease-service

The Kubernetes Service provides network access to the deployed pods. A LoadBalancer service type was used to expose the API externally through port 80, enabling users to access the prediction endpoint using the assigned Kubernetes URL.

8.4. Deployment Verification

The successful Kubernetes deployment was verified by checking the running pods, service status, and API response.

The screenshot displays a terminal window with a light green header. The main content is a dark gray area containing a curl command and its output. The curl command is a POST request to `http://127.0.0.1:58618/predict` with headers for `accept: application/json` and `Content-Type: application/json`. The request body is a JSON object with patient data. The response is a 200 status code with a JSON body containing `heart_disease_risk: 1` and `confidence: 0.5961426620055138`.

```
Curl
curl -X 'POST' \
  'http://127.0.0.1:58618/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "age": 65,
    "sex": 1,
    "cp": 2,
    "trestbps": 134,
    "chol": 200,
    "fbs": 1,
    "restecg": 1,
    "thalach": 120,
    "exang": 1,
    "oldpeak": 2.5,
    "slope": 1,
    "ca": 2,
    "thal": 3
  }'
```

Request URL

```
http://127.0.0.1:58618/predict
```

Server response

Code	Details
200	Response body <pre>{ "heart_disease_risk": 1, "confidence": 0.5961426620055138 }</pre>

Figure 20: Successful heart disease prediction request through the Kubernetes exposed API endpoint.

9. Monitoring & Logging

9.1. Monitoring Overview

Monitoring is an essential component of the MLOps pipeline because it provides visibility into the performance, reliability, and health of the deployed machine learning API. In this project, Prometheus was used to collect application metrics, while Grafana was used to visualize these metrics through interactive dashboards.

The monitoring solution enables continuous observation of the deployed FastAPI service, helping identify performance issues, API failures, and abnormal behavior during operation.

9.2. Prometheus Monitoring

The FastAPI application was instrumented with Prometheus to expose operational metrics through a dedicated endpoint. The application exposes the following endpoint: `/metrics`. Prometheus periodically collects metrics from this endpoint and stores them for analysis.

The monitored metrics include:

HTTP Request Count: Tracks the total number of API requests received and helps measure API usage and traffic.

Request Latency: Measures the response time required to process prediction requests and is useful for identifying performance bottlenecks.

Error Count: Tracks failed API requests and server errors and helps detect failures and maintain service reliability.

These metrics provide continuous insight into the operational performance of the deployed heart disease prediction service.

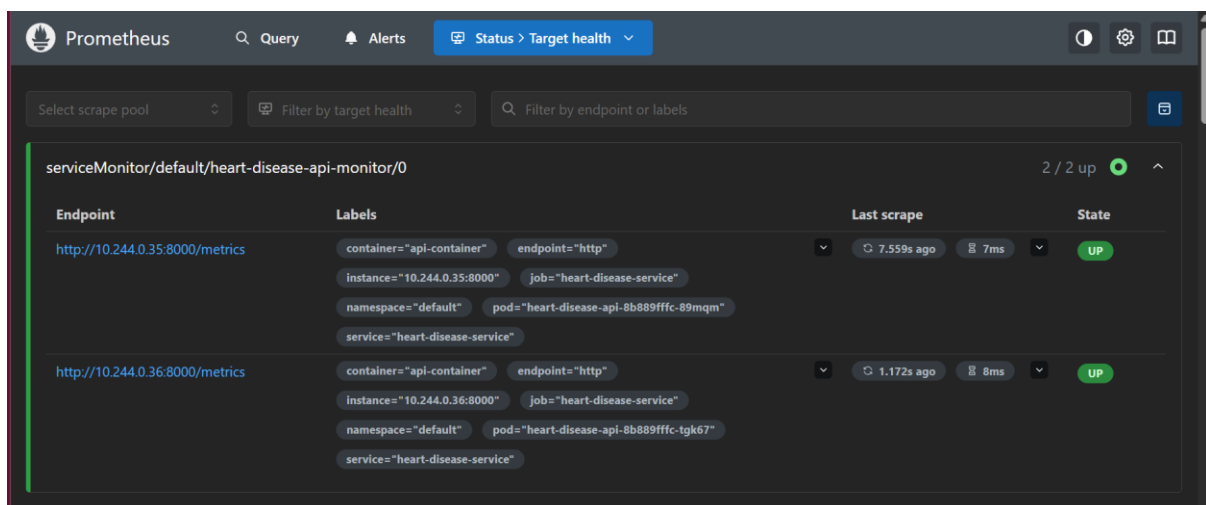


Figure 21: Prometheus successfully scraping metrics from the heart disease API.

9.3. Grafana Dashboard

Grafana was integrated with Prometheus to visualize application metrics using interactive dashboards. The dashboard provides a real-time view of API performance and availability, making it easier to monitor the deployed service.

The dashboard includes visualizations for:

- **HTTP Request Count**
 - Displays the number of prediction requests received over time.
- **Response Duration (Latency)**
 - Shows the response time for API requests, helping identify slow responses.
- **API Availability and Errors**
 - Displays service health and the number of failed requests, allowing quick detection of operational issues.

These visualizations help monitor the reliability and responsiveness of the heart disease prediction API in a production environment.



Figure 22: Grafana dashboard visualizing request rate, latency, and API error metrics collected by Prometheus.

10. Conclusion & Future Improvements

The project focuses on developing a complete MLOps pipeline for a heart disease prediction system, covering the entire machine learning lifecycle from data acquisition and exploratory analysis to deployment and monitoring.

The current implementation includes data preprocessing, feature engineering, model development using classification algorithms, experiment tracking with MLflow, automated model training, reproducible model packaging, CI/CD integration with automated testing, Docker-based model serving, and monitoring of deployed APIs. The pipeline ensures reliable and repeatable workflows by maintaining consistent environments, tracking experiments, and enabling automated deployment processes.

Future improvements include migrating the deployment infrastructure to cloud platforms such as AWS, GCP, or Azure, integrating a centralized model registry for better model version management, implementing automated model retraining pipelines, and adding advanced data drift detection mechanisms to maintain model performance over time.